



Tareas tema 4, tercera parte.

Para el tema 4.3, deberéis entregar los ejercicios propuestos a continuación. No es necesario que os ciñáis al pie de la letra al enunciado, podéis cambiar a vuestro gusto siempre que se adapte a lo que se pide (o se mejore).

La entrega constará del código fuente sin archivos temporales, **y capturas de pantalla** tanto de la aplicación como de las imágenes generadas (los archivos temporales que tenéis que borrar son las carpetas obj y x64 que hay en todas las subcarpetas del proyecto). Mejor si es un documento PDF con las capturas de pantalla y explicaciones del (o de los) programas desarrollados.

Los ejercicios estan relacionados con la tercera parte del tema 4 sobre (Pre)Procesado de Imágenes, aunque como siempre, podéis usar como base los programas que ya tenéis hechos de los temas anteriores o que vayamos haciendo en las clases. En principio a idea es hacerlos con interfaz gráfica, pero si alguien prefiere hacerlos por línea de comando, no hay problema. También pueden ser también varios programas, o uno solo que tenga todas las opciones, a vuestro gusto. En la entrega comentad un poco la solución adoptada.

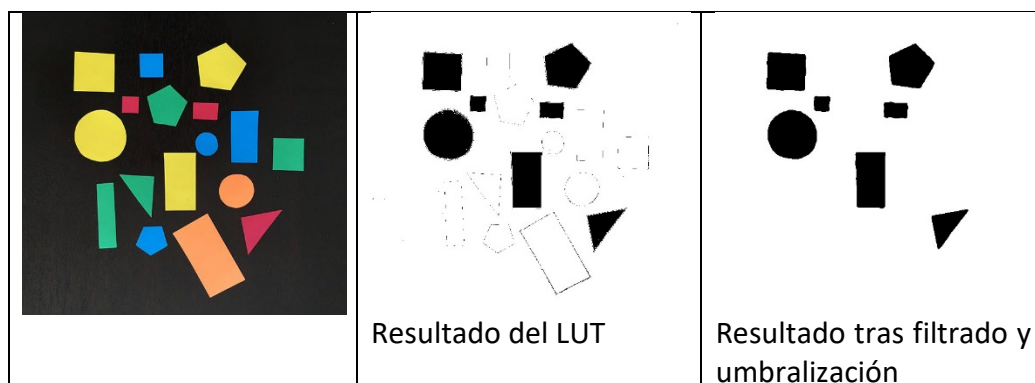
Estos son los ejercicios a hacer:

1. Ejercicio 1: Utilizando las funciones de OpenCV calcular la media del brillo, y la desviación (como medida del contraste). Imprimir estos valores sobre la imagen cargada con las funciones de texto de OpenCV. Hacer que el cálculo se pueda realizar sobre toda la imagen, o sobre una imagen y su máscara ¿Cuáles son estos valores?
2. Ejercicio 2: Hacer un ejercicio donde tengan que aplicar un LUT. En concreto, aplicar un LUT para umbralizar los colores rojo y amarillo de la imagen shapes_and_colors.jpg. Se puede hacer la conversión previa a niveles de gris, ya que, en este plano, aún se aprecian diferencias entre los colores. Por tanto, se trata de implementar un LUT del tipo:



Es decir, todo lo que no sea rojo o amarillo se pone a 255, y los píxeles con ese color, se ponen a 0.

(aplicar una gaussiana y una umbralización por 128 a la imagen resultante del LUT para eliminar ruido en los bordes de otros objetos.





3. Ejercicio 3: Añadir una función para realizar una ecualización del histograma, y que realice una ecualización en la imagen en niveles de gris, pero si es una imagen de color, se realice una ecualización del canal H (espacio de color HSV). Probarlo con las imágenes `img_066_ROI.png` e `img_066-ROI-G.png`.
4. Ejercicio 4: Programar la aplicación de filtros de Sobel de forma que se pueda escoger derivadas de 0 (no aplicar Sobel en el sentido indicado), 1 y 2 orden en x e y, y el tamaño del kernel de 3x3 o 5x5). Adjuntar los resultados de aplicar `(dx,dy,ksize)` de `(0,1,3)``(1,0,3)``(1,1,3)` y `(2,2,5)` a la imagen `screen_pattern.jpg`
5. Ejercicio 5: Implementar el filtro de Canny. Implementarlo con barras de slide que permitan cambiar los parámetros de umbral, y observar la influencia sobre la imagen, por ejemplo, las anteriores.
6. Ejercicio gordo: Queremos aplicar un suavizado sobre una imagen donde hay personas para la protección de testigos (o menores, o policía, etc.). Partimos de una imagen (`crow_people.jpg`), y una máscara que determina el rectángulo con la posición de la cabeza (`crow_people_facemask.png`) aún no tenemos acceso a aplicaciones de reconocimiento de caras). Aplicar un suavizado que permita “ocultar” o “difuminar” suficientemente únicamente la cara fijada por la máscara.

Nota: no existen las máscaras en las operaciones de filtrado. Es posible que sea necesario utilizar operaciones aritméticas, y que además de la imagen `crow_people.jpg`, sea necesario también su inversa.

